PATRÓN DE DISEÑO

DECORADOR

Presentado por:
Giovanni Hernández Sánchez

Tabla de contenido

01 Nombre y propósito

02 Motivación

03 Aplicabilidad

04 Estructura

05 Participantes

06 Colaboraciones

07 Consecuencias

08 Implementación

09 Usos conocidos

10 Patrones relacionados

11 Ejemplo

12 Actividad

Patrón:

Decorador (Decorator)

Categoría:

Estructural de objetos

Propósito:

Asigna responsabilidades adicionales a un objeto dinámicamente, proporcionando una alternativa flexible a la herencia para extender la funcionalidad.

También conocido como: ***Wrapper (Envoltorio)***

Cuando tienes frío, te cubres con un suéter. Si sigues teniendo frío a pesar del suéter, puedes ponerte una chaqueta encima. Si está lloviendo, puedes ponerte un impermeable.



Figura 1. Patrón decorador en la vida real. (Fuente: Refactoring Guru, “Decorator”, refactoring.guru, 2025)

Motivación

Añadir responsabilidades a objetos individuales, no a toda una clase.

Este patrón brinda un enfoque flexible al encerrar el componente o clase en otro objeto que añada un comportamiento dinámico. Al objeto confinante se le denomina ***decorador****.*

Por ejemplo, un toolkit de interfaces de usuario debería permitir añadir propiedades (como bordes) o comportamientos (como capacidad de desplazamiento) a cualquier componente de la interfaz de usuario.

Ejemplo:

- Añadir un DecoradorDesplazamiento para barras de scroll.
- También un DecoradorBorde para bordes.

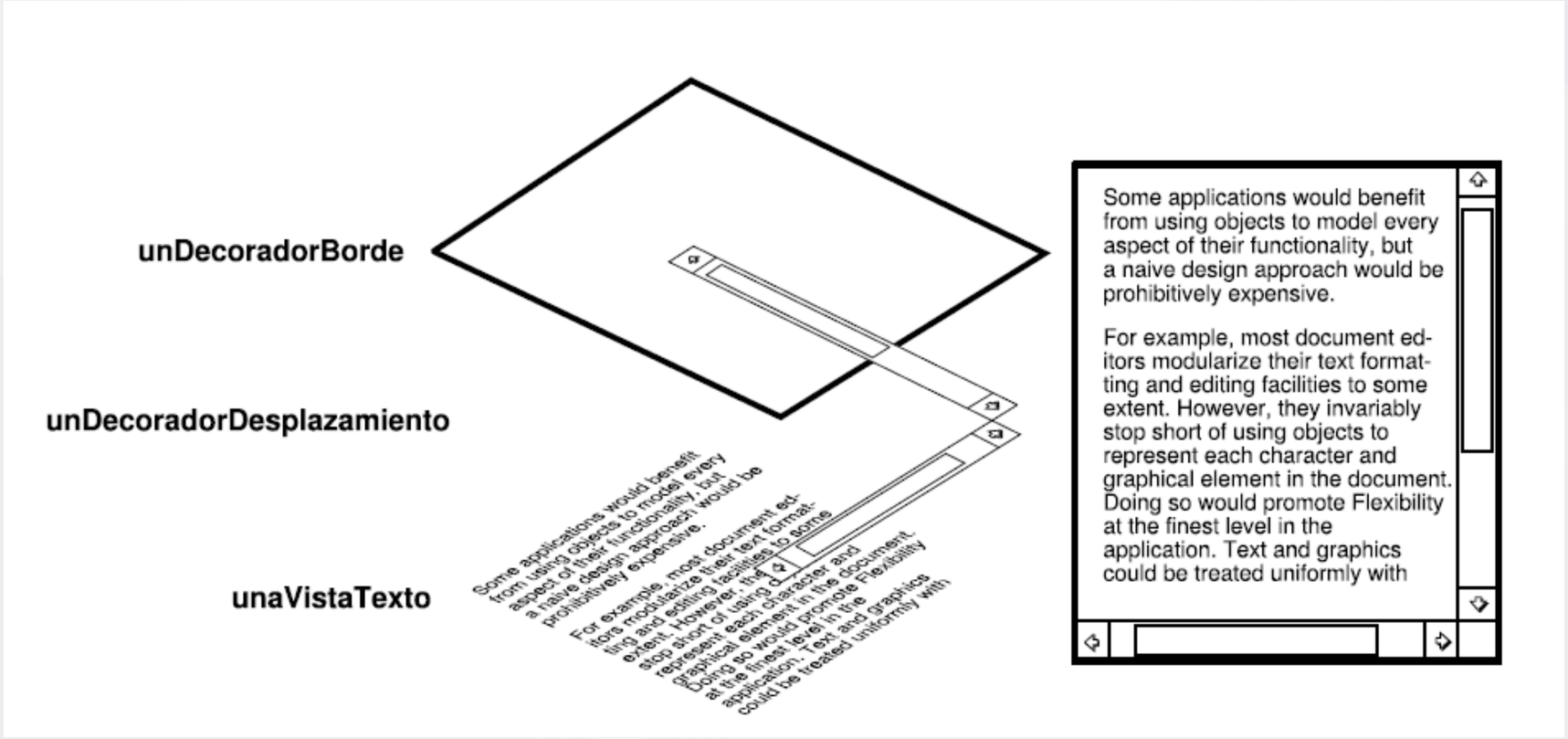


Figura 2. Aplicación del patrón Decorador. (Fuente: Gamma, Helm, Johnson & Vlissides, 1994, Design Patterns: Elements of Reusable Object-Oriented Software).

Aplicabilidad

Usar el patrón cuando:

*Se necesite añadir responsabilidades de manera **dinámica y transparente**.*

Las responsabilidades puedan retirarse fácilmente.

*La **herencia no sea viable** o produzca demasiadas subclases*

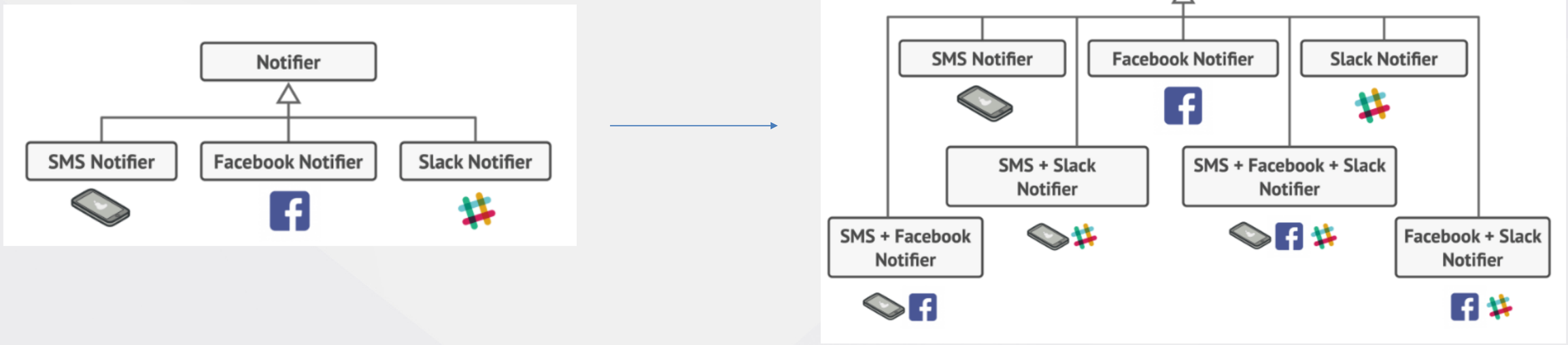


Figura 3. Problemas de aumento de subclases. (Fuente: Refactoring Guru, “Decorator”, refactoring.guru, 2025)

Estructura

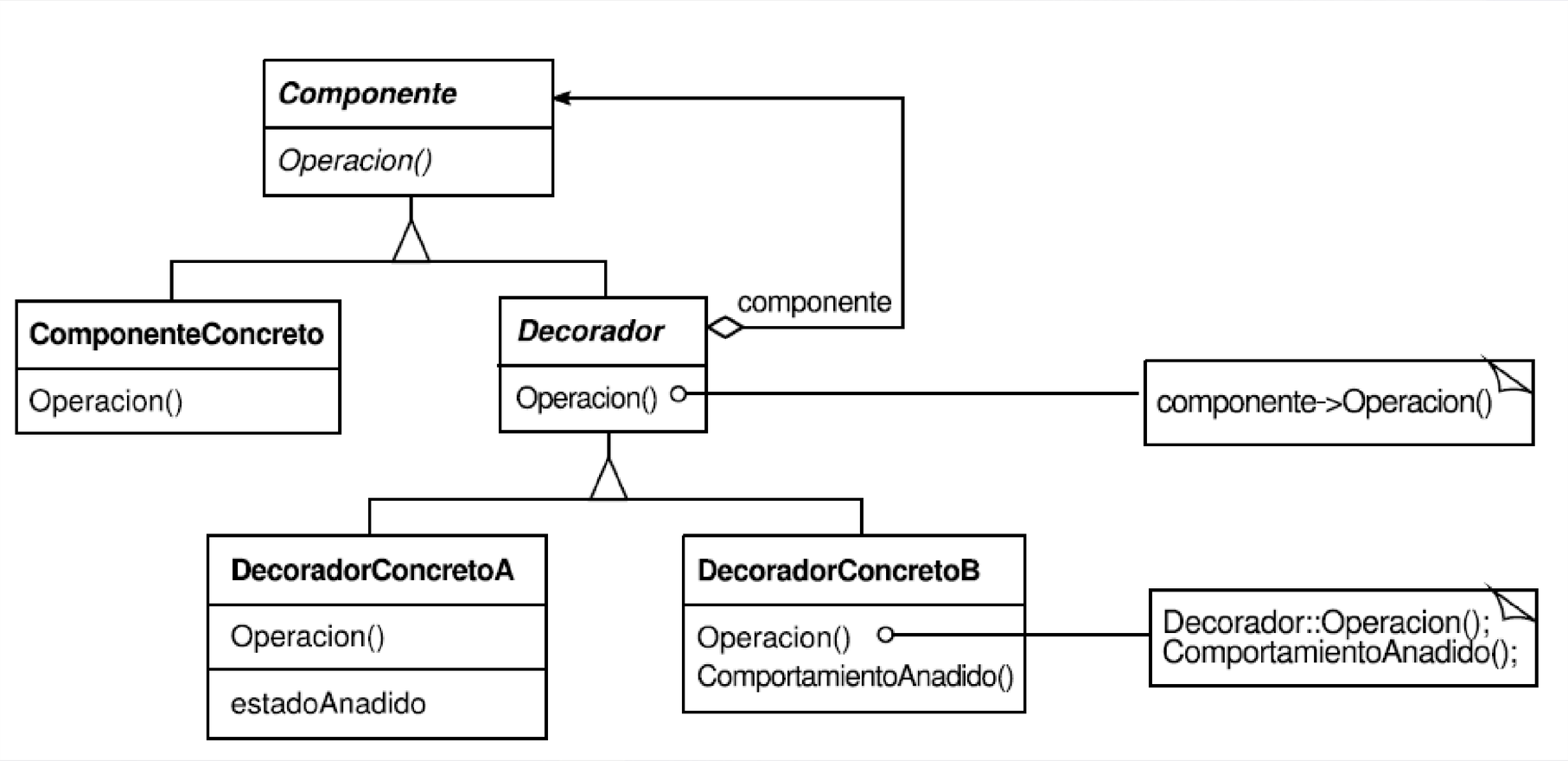


Figura 4. Estructura del patrón Decorador. (Fuente: Gamma, Helm, Johnson & Vlissides, 1994, Design Patterns: Elements of Reusable Object-Oriented Software).

Participantes

Componente:

Define la interfaz para objetos a los que se puede añadir responsabilidades dinámicamente.

Decorador:

Mantiene una referencia a un objeto Componente y define una interfaz que se ajusta a la interfaz del Componente.

Componente Concreto:

Define un objeto al que se pueden añadir responsabilidades adicionales.

Decorador Concreto:

Añade responsabilidades al componente.

Colaboraciones

- *El Decorador redirige peticiones a su objeto Componente.*
- *Opcionalmente puede realizar operaciones adicionales antes y después de reenviar la petición.*

Consecuencias

Ventajas

- *Más flexibilidad que la herencia estática.*
- *Evita clases sobrecargadas.*

Desventajas

- *Un decorador y su componente no son idénticos.*
- *Puede producir muchos objetos pequeños.*

Implementación

01.

La interfaz del decorador debe coincidir con la del componente.

02.

Se puede omitir la clase abstracta si no es necesaria.

03.

Mantener ligera la clase base (Clase común entre el decorador y componentes).

04.

Decorador cambia el exterior, no el interior (diferencia con Strategy).

Usos conocidos

Interfaces Gráficas de Usuario (GUI):
Se puede usar para añadir bordes o marcos a campos de texto.

Flujos de Datos (Streams):
Añadir propiedades e información de estado a los flujos de datos.

Sistemas de Notificaciones:

Permite combinar y expandir funcionalidades de envío de notificaciones.

Funciones de reintento:
Reintento automático o de almacenamiento en caché en redes simples.

Patrones relacionados

Adapter

Decorador sólo cambia las responsabilidades de un objeto, mientras que un adaptador una interfaz completamente nueva.

Strategy

Un decorador permite cambiar el exterior de un objeto; una estrategia permite cambiar sus tripas.

Composite

Un decorador es como un compuesto que sólo tiene un componente. No obstante, un decorador sólo añade responsabilidades adicionales.

Ejemplo

Cafetería

Se tiene una cafetería que ofrece varios tipos de café (Simple, con leche y con azúcar).
El ejemplo consiste en un café simple al que se le pueden añadir dinámicamente diferentes ingredientes adicionales, como leche o azúcar. Cada ingrediente se modela como un decorador que incrementa el costo y modifica la descripción del café. De esta manera, es posible combinar decoradores en cualquier orden para obtener distintas variantes de café sin necesidad de crear múltiples subclases.

Ejemplo

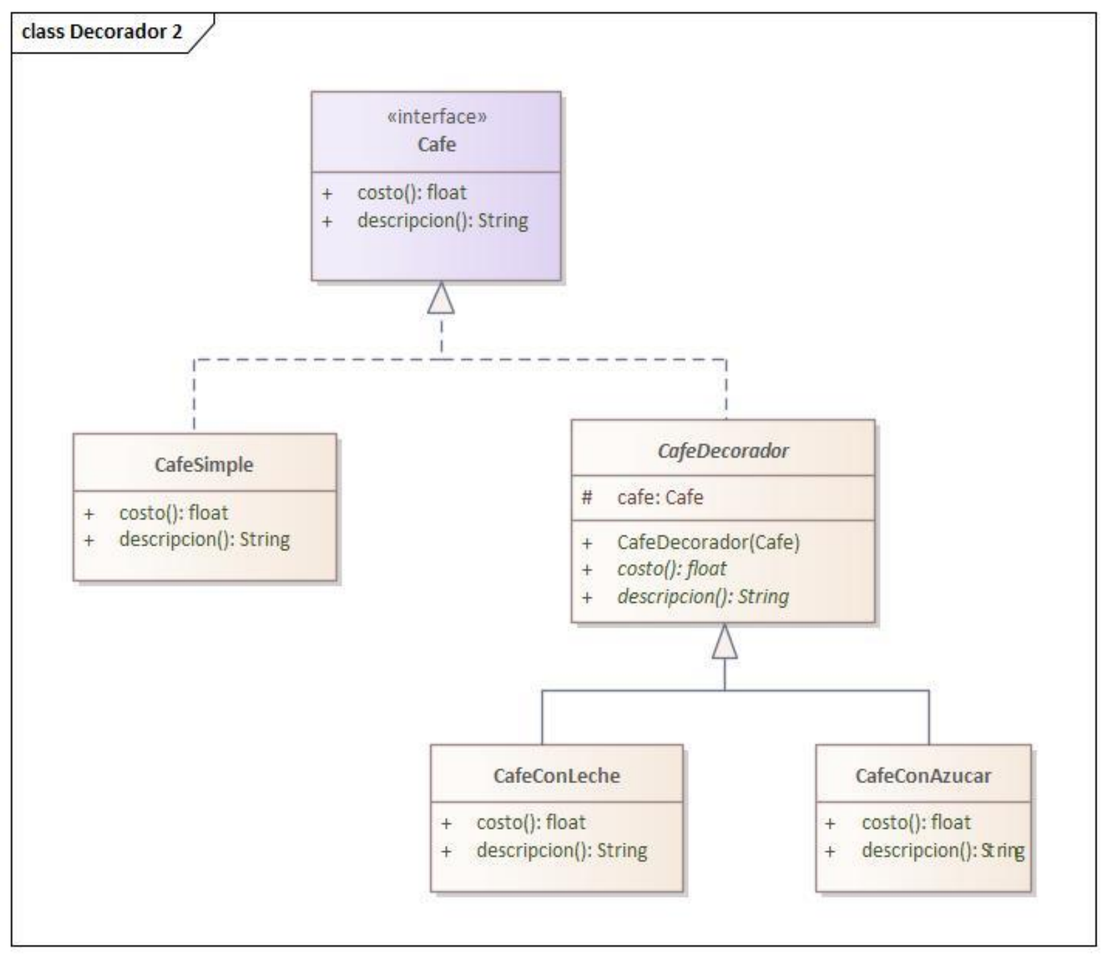


Figura 5. Diagrama de clases de un ejemplo de la aplicación del patrón Decorador. (Fuente: Propia).

Actividad en clase

Enunciado:

Se tiene una interfaz auto que define un método costo y descripción.
Un auto básico cuesta $20,000. Se quiere añadir accesorios (aire acondicionado, radio, sensor de reversa, etc.) de manera dinámica sin crear muchas subclases.

Actividad:

Diseña un diagrama de clases UML que represente el patrón Decorador aplicado a este escenario:

- Auto como componente (interfaz).
- AutoBasico como componente concreto.
- AutoDecorador como decorador abstracto.
- ConAireAcondicionado, ConRadio, ConSensorReversa como decoradores concretos.

MUCHAS GRACIAS

Presentado por:
Giovanni Hernández Sánchez

Universidad de Cartagena
Facultad de Ingeniería
Ingeniería de Sistemas
VIII Semestre
Arquitectura de Software

Referencias bibliografías

[1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[2] Refactoring.Guru. (2025). Patrón Decorator. Recuperado de https://refactoring.guru/es/design-patterns/decorator